PROJET LC-3

1/ Changements du circuit original

DecodeIr : Nous avons redirigé les 4 bits de poids fort (op-code) de l'instruction en cours d'exécution (register instruction) vers les sorties correspondantes. Pour le module arithmétique nous avons juste les bits b^{12} et b^{13} à redirigé car ce sont les seuls bits commun aux opcode représentant les opérations arithmétique et elles sont unique dans la table d'opcode du cours, dans le circuit, le bit 12 est à True et le bit 13 est à False (raisonnement analogue pour l'unité de chargement avec le bit 12 à False et le bit 13 à True). Ensuite, pour l'ensemble des instructions Jump et Store nous avions besoin des 4 bits de poids fort car ils n'ont aucun ensemble de bit de poids fort en commun pour représenter de manière unique l'opcode. Nous avons donc redirigé les 4 bits en fonction des valeurs de chacun pour les Load et les Store. Nous avons, par ailleurs, fait une disjonction de 3 sorties pour savoir quand nous devons actualiser l'instruction NZP. Étant donné que seules les opérations arithmétiques et du chargement modifient les flags de saut, il à fallu les rediriger dans cette porte OR pour pouvoir indiquer quand écrire dans les flags de BR.

Alu : Ici nous n'avons qu'ajouter l'instruction de popcnt que nous expliquerons plus bas et l'ajout du Xor tout en gardant le Not actif. Ceci à été possible car le Not est un cas particulier du Xor. En regardant le codage de l'instruction Xor nous nous rendons compte que le Not est exactement le Xor avec l'immédiat rempli de 1. Cela revient à faire le xor bit à bit de ce même entier avec n bits à 1. Le not étant l'opposé de chaque bit, dans le xor si nous avons un 1 alors cela donnera 0 tandis que si nous avons un 0 cela nous donnera 1 car XOR(1,0) = 1 et XOR(1,1)=0.

PopCnt : Nous avons implémenté le popcount en subdivisant le circuit en 3 sous circuits qui sont respectivement popcnt2, popcnt4 et popcnt8. Pour pouvoir avoir une profondeur logarithmique (4 portes AND pour 16 bits) et non linéaire (donc 16 portes AND pour 16 bits). Nous avons dû utiliser plusieurs splitter afin d'"étendre" le nombre de bits d'un entier.

WriteVal : Nous avons juste suivi les indications marquées. Nous avons vu en cours que le MUX peut être utilisé comme une condition de programmation et comme l'indication était formulé comme telle, nous avons rediriger les sorties en fonction des valeurs présentes. AluRes étant la sortie par défaut, MemOut est à 1 car la sortie est MemOut uniquement quand Load est à 1 donc c'est pratique, LEA vaut 2 donc Load + Lea, on envoie PC quand JSR vaut 1 (avec l'extension de base, cela revient à 11=3). En ce qui concerne l'incrément du PC, nous avons besoin de l'incrémenter de 1 car en exécutant le circuit, nous remarquons que l'instruction à 1 position de retard en mémoire.

NZP : Pour ce qui concerne le circuit NZP, nous modifions le résultat du calcul uniquement quand nous avons une opération qui modifient les flags car nous devons garder en mémoire le même résultat tant qu'on à pas de nouvelle instruction modifiant les flags. L'instruction courante précise quel flag tester pour le saut conditionnelle. Nous combinons cela avec le résultat des flags pour savoir si on saute fait effectivement le testNZP.

2/ Programmes de test

Popcnt.asm :

```
   .ORIG x3000
   AND R0, R0, 0
   ADD R0, R0, 15
   AND R1, R1, 0
   POPCNT R1, R0
   .END
```

L'exécution de ce code retourne 4 car nous mettons dans le registre R1 le nombre de 1 contenu dans le registre R0 qui vaut 15

Xor.asm :

```
   .ORIG x3000
   AND R0, R0, 0
   AND R1, R1, 0
   ADD R0, R0, 8
   ADD R1, R1, 5
   XOR R2, R0, R1
   .END
```

Ici, nous utilisons presque tout le module arithmétique. La valeur stockée dans le registre R2 est le xor bit à bit entre R0 (=8) et R1 (=5). La valeur stockée dans le registre R2 est 1101 (=13).

Not.asm :

```
   .ORIG x3000
   AND R0, R0, 0
   ADD R0, R0, 4
   NOT R0, R0
   .END
```

Ici nous stockons dans NOT le NOT de $(0000\ 0000\ 0000\ 0100)^2$ qui donne $(1111\ 1111\ 1111\ 1011)^2$

Br2.asm :

```
       AND R0, R0, 0
       AND R1, R1, 0
       LEA R1, test
loop:   LDR R0, R1, 0
       BRz fin
       ADD R1, R1, 1
       BR loop
```

```
test:    .STRINGZ "p"
fin:     .END
```

Ce programme va simplement parcourir la boucle et s'arrêter quand il aura fini de la parcourir. Il n'y a pas de sortie particulière mais l'exécution du programme doit stocker dans R0 toutes les adresses des char de test.

ldr.asm :

```
       .ORIG x3000
       AND R0, R0, 0
       AND R1, R1, 0
       LEA R1, test
       LDR R0, R1, 0 ;R0 devrait donc être égale à 18
test:   .FILL 18
       .END
```

Ce programme devrait donc stocker dans R0 l'entier binaire représentant 18 sur 16 bits soit 0000 0000 0001 0000